AI-Powered Chatbot Web Assignment

Problem Statement

Build a full-stack AI-powered chatbot application using **Next.js (App Router)** and the **Vercel AI SDK**. Integrate with any AI provider of your choice (recommended: Fireworks.ai, Groq, OpenAI, Anthropic, or Together.ai) to enable real-time streaming responses from a language model. The application must feature a clean, modern chat interface with real-time message streaming and persistent conversation history within the session.

Core Features

1. **Chat Interface**
    - Display a scrollable message list showing both user and assistant messages (visually distinguish with alignment, colors, and optional avatars).
    - Include a text input field at the bottom with a send button
    - Use Vercel AI SDK's streamText (or generateText with streaming) to stream AI responses in real-time (text should appear progressively).
    - Show a typing/loading indicator while the model is generating a response.
2. **Conversation History**
    - Maintain full conversation history for the current session (all messages persist as the user continues chatting).
    - History must be preserved on page refresh (implement using localStorage or server-side session if preferred).
    - Display the entire history on load/refresh without losing context.
3. **AI Provider Integration**
    - Use Vercel AI SDK to handle streaming and provider abstraction.
    - Choose and configure one provider and model (e.g., Fireworks accounts/fireworks/models/llama-v3p1-70b-instruct, Groq llama3-8b-8192, etc.).
    - Store API key securely in .env.local and access only in server-side code (API routes or server actions).
    - Include basic error handling (e.g., display user-friendly message on API failure).

UI Expectations

- Clean, modern chat UI inspired by ChatGPT or Claude (minimalist design, proper spacing, readable typography).
- Fully responsive across mobile, tablet, and desktop devices.
- Smooth animations and transitions (e.g., message fade-in, typing indicator dots).

- Thoughtful UX: disable send button when input is empty or streaming, auto-focus input on load, handle empty states gracefully.

Bonus Features (Optional)

- “New Chat” button to clear history and start a fresh conversation.
- Model selection dropdown (switch between multiple models/providers).
- Copy button for individual messages.

Submission Instructions

- Push the complete code to a public GitHub repository.
- Include both frontend and any backend/API routes in the same repo (Next.js app).
- Provide clear setup instructions in the README
- Ensure the app runs without errors on modern browsers.

Estimated Time

6 Hours

Evaluation Criteria

Criteria	Weight
Code Quality & Folder Structure	20%
Streaming & AI Integration Logic	20%
UI Design & User Experience	20%
Functionality (Chat & History)	20%
Responsiveness & Performance	10%
Security (API Key Handling)	10%